CP03 PROGRAMMING & SOFTWARE DEVELOPMENT

PYTHON CHEAT SHEET

Beginner's Quick Reference

Essential Python syntax untuk mula coding dengan lebih yakin.

```python
def greet(name):
    print(f"Hi {name}")

greet("Yusuf")
# Hi Yusuf
```

01 Basic Python Syntax

Python guna **indentation** untuk tentukan code block. Tiada kurungan `{ }`.

INDENTATION

```python
if True:
    print("Hello, Python!")
```

COMMENT

```python
# This is a comment
print("Hello World")
```

PENTING

- Indentation matters: salah space, program error.
- Colon `:` memperkenalkan indented code block.
- Python is case-sensitive: `name` dan `Name` dua variable berbeza.
- Comment bermula dengan `#`.

02 Variables

A variable ialah nama yang digunakan untuk merujuk kepada sesuatu value.

SYNTAX

```python
name = "Yusuf"
age = 21
height = 158.5
is_student = True
```

`name` simpan string
`age` simpan integer
`height` simpan floating-point number
`is_student` simpan Boolean

VARIABLE NAMING RULES

- Mula dengan huruf atau underscore.
- Tidak boleh mula dengan nombor.
- Boleh ada huruf, nombor dan underscore.
- Case-sensitive.
- Tidak boleh guna Python reserved keywords.

GOOD EXAMPLES

```python
student_name = "Ali"
total_price = 50
user_age = 20
```

BAD EXAMPLES

```python
2name = "Ali"       # Invalid
student-name = "Ali" # Invalid
```

03 Data Types

Data type menentukan jenis value yang disimpan: nombor, teks, collection dan sebagainya.

DATA TYPE	DESCRIPTION	EXAMPLE
int	Whole numbers	`25`
float	Decimal numbers	`19.99`
str	Text	`"Hello"`
bool	True or False	`True`
list	Ordered, mutable collection	`[1, 2, 3]`
tuple	Ordered, immutable collection	`(1, 2, 3)`
dict	Key-value collection	`{"name": "Ali"}`
set	Collection of unique elements	`{1, 2, 3}`
NoneType	Represents no value	`None`

CHECK DATA TYPE

```python
age = 21
print(type(age))
```

OUTPUT

```
<class 'int'>
```

TYPE CONVERSION

```python
age = int("21")
price = float("19.90")
number = str(100)
```

PENTING

- `input()` return string by default.
- Convert input bila perlukan kiraan nombor.
- Jangan keliru `"10"` (string) dengan `10` (integer).

04 Strings

String ialah data type yang digunakan untuk menyimpan teks.

CREATING STRINGS

```
name = "Yusuf"
message = 'Hello World'
```

STRING OPERATIONS

```
text = "Python Programming"

print(text.lower())
print(text.upper())
print(text.title())
print(text.strip())
print(text.replace("Python", "Java"))
```

OUTPUT: python programming | PYTHON PROGRAMMING | Python Programming | Python Programming | Java Programming

ACCESSING CHARACTERS

```
language = "Python"

print(language[0])   # P
print(language[-1])  # n
```

STRING SLICING

```
language = "Python"

print(language[0:3])  # Pyt
print(language[2:])   # thon
print(language[:4])   # Pyth
```

STRING LENGTH

```
name = "Yusuf"
print(len(name))
```

OUTPUT

```
5
```

STRING FORMATTING (F-STRING)

```
name = "Yusuf"
age = 21

print(f"My name is {name} and I am {age}.")
```

PENTING

- Strings are **immutable**: tidak boleh diubah selepas dicipta.
- Indexing bermula dari **zero**.
- Negative indexing bermula dari hujung (`-1` = aksara terakhir).
- Ending position dalam slice **dikecualikan**: `[0:3]` ambil index 0, 1, 2.
- Guna f-string sebagai kaedah formatting utama.

05 Lists

List ialah collection yang menyimpan beberapa items dalam satu variable. List adalah ordered dan mutable.

CREATING A LIST

```
fruits = ["Apple", "Banana", "Orange"]
```

ACCESSING ITEMS

```
print(fruits[0])
print(fruits[-1])
```

OUTPUT

```
Apple
Orange
```

MODIFYING ITEMS

```
fruits[1] = "Mango"
```

ADDING ITEMS

```
fruits.append("Grapes")
fruits.insert(1, "Watermelon")
```

REMOVING ITEMS

```
fruits.remove("Apple")
fruits.pop()
del fruits[0]
```

USEFUL LIST METHODS

```
numbers = [3, 1, 2]

numbers.sort()
numbers.reverse()

print(len(numbers))
```

LOOP THROUGH A LIST

```
for fruit in fruits:
    print(fruit)
```

OUTPUT

```
Mango
Orange
```

PENTING

- Lists are ordered dan mutable.
- Lists allow duplicate values.
- Indexing starts at zero.

06 Dictionaries

Dictionary menyimpan data dalam bentuk key-value pairs.

CREATING A DICTIONARY

```
student = {
    "name": "Ali",
    "age": 21,
    "course": "IT"
}
```

ADDING AND UPDATING

```
student["age"] = 22
student["university"] = "MMU"
```

REMOVING ITEMS

```
student.pop("course")
```

ACCESSING VALUES

```
print(student["name"])
print(student.get("age"))
```

LOOP THROUGH DICTIONARY

```
for key, value in student.items():
    print(key, value)
```

Key tiada: `student["name"]` raise KeyError, `.get()` return `None` by default.

PENTING

- Keys must be unique.
- Dictionaries are mutable.
- Value diakses guna key, bukan position macam list.

07 Conditional Statements

Conditional statements membolehkan program membuat keputusan berdasarkan sesuatu condition.

IF

```
age = 20

if age >= 18:
    print("Adult")
```

IF ELSE

```
age = 16

if age >= 18:
    print("Adult")
else:
    print("Minor")
```

IF ELIF ELSE

```
score = 85

if score >= 90:
    print("A")
elif score >= 80:
    print("B")
elif score >= 70:
    print("C")
else:
    print("Fail")
```

COMPARISON OPERATORS

OPERATOR	MEANING
`==`	Equal
`!=`	Not equal
`>`	Greater than
`<`	Less than
`>=`	Greater than or equal
`<=`	Less than or equal

LOGICAL OPERATORS

OPERATOR	MEANING
`and`	Both conditions must be True
`or`	At least one condition must be True
`not`	Reverses Boolean value

LOGICAL EXAMPLE

```
age = 22
has_id = True

if age >= 18 and has_id:
    print("Access granted")
```

MEMBERSHIP OPERATORS

```
fruits = ["Apple", "Banana"]

print("Apple" in fruits)
print("Orange" not in fruits)
```

OUTPUT

```
True
True
```

PENTING

- Guna `==` untuk compare values, `=` untuk assign values.
- Conditions evaluate kepada Boolean results.
- Indentation menentukan statement block.

08 Loops

Loop digunakan untuk menjalankan sesuatu block of code berulang kali.

FOR LOOP

```
for i in range(5):
    print(i)
```

OUTPUT

```
0
1
2
3
4
```

`range(5)` hasilkan value dari 0 sehingga 5, tetapi 5 tidak termasuk.

RANGE EXAMPLES

```
range(5)        # 0 to 4
range(1, 6)     # 1 to 5
range(0, 10, 2) # 0, 2, 4, 6, 8
```

WHILE LOOP

```
count = 1

while count <= 5:
    print(count)
    count += 1
```

OUTPUT

```
1
2
3
4
5
```

While loop jalan selagi condition True. Update variable dengan betul supaya tidak jadi infinite loop.

BREAK

```
for i in range(10):
    if i == 5:
        break

    print(i)
```

OUTPUT

```
0
1
2
3
4
```

`break` hentikan loop serta-merta.

CONTINUE

```
for i in range(5):
    if i == 2:
        continue

    print(i)
```

OUTPUT

```
0
1
3
4
```

`continue` langkau pusingan semasa.

LOOP THROUGH LIST

```
names = ["Ali", "Abu", "Amin"]

for name in names:
    print(name)
```

09 Functions

Function ialah reusable block of code yang menjalankan tugas tertentu.

BASIC FUNCTION

```
def greet():
    print("Hello!")

greet()  # call the function
```

OUTPUT

```
Hello!
```

FUNCTION WITH PARAMETERS

```
def greet(name):
    print(f"Hello, {name}!")

greet("Yusuf")
```

OUTPUT

```
Hello, Yusuf!
```

`name` ialah parameter. `"Yusuf"` ialah argument.

FUNCTION WITH RETURN

```
def add(a, b):
    return a + b

result = add(5, 3)
print(result)
```

OUTPUT

```
8
```

`return` hantar value kembali kepada caller. `print()` hanya paparkan output. Dua fungsi berbeza.

DEFAULT PARAMETERS

```
def greet(name="Guest"):
    print(f"Hello, {name}!")

greet()
greet("Ali")
```

OUTPUT

```
Hello, Guest!
Hello, Ali!
```

MULTIPLE PARAMETERS

```
def calculate_total(price, quantity):
    return price * quantity

total = calculate_total(10, 3)
```

QUICK FUNCTION SYNTAX

```
def function_name(parameters):
    # code
    return value
```

PENTING

- Guna `def` untuk takrif function. Code dalam function perlu indentation.
- Function mesti ditakrif sebelum dipanggil.
- Guna `snake_case` untuk nama function dan variable.

10 Input & Output

Input membolehkan program menerima data daripada pengguna, manakala output memaparkan hasil.

PRINT OUTPUT

```
print("Hello World")
print(100)
print("Total:", 50)
```

OUTPUT

```
Hello World
100
Total: 50
```

USER INPUT

```
name = input("Enter your name: ")

print(f"Hello, {name}!")
```

NUMERIC INPUT

```
age = int(input("Enter your age: "))
price = float(input("Enter price: "))
```

`input()` return string, jadi guna `int()` atau `float()` bila perlukan nilai nombor.

SIMPLE CALCULATION

```
num1 = float(input("First number: "))
num2 = float(input("Second number: "))

total = num1 + num2

print(f"Total: {total}")
```

OUTPUT FORMATTING

```
price = 19.9

print(f"Price: RM{price:.2f}")
```

OUTPUT

```
Price: RM19.90
```

PENTING

- `.2f` format value kepada dua decimal places.
- f-strings bagi variable interpolation yang mudah dibaca.

11 Common Python Operators

Operator ialah simbol untuk buat kiraan atau tukar value variable.

ARITHMETIC OPERATORS

OPERATOR	MEANING	EXAMPLE
`+`	Addition	`5 + 2`
`-`	Subtraction	`5 - 2`
`*`	Multiplication	`5 * 2`
`/`	Division	`5 / 2`
`//`	Floor division	`5 // 2`
`%`	Modulus	`5 % 2`
`**`	Exponentiation	`5 ** 2`

ASSIGNMENT OPERATORS

OPERATOR	EXAMPLE
`=`	`x = 10`
`+=`	`x += 2`
`-=`	`x -= 2`
`*=`	`x *= 2`
`/=`	`x /= 2`

12 Common Built-in Functions

Built-in functions boleh terus digunakan tanpa perlu import apa-apa.

FUNCTION	PURPOSE	EXAMPLE
`print()`	Display output	`print("Hello")`
`input()`	Receive user input	`input("Name: ")`
`type()`	Check data type	`type(10)`
`len()`	Get length	`len("Python")`
`int()`	Convert to integer	`int("10")`
`float()`	Convert to float	`float("2.5")`
`str()`	Convert to string	`str(100)`
`bool()`	Convert to Boolean	`bool(1)`
`range()`	Generate integer sequence	`range(5)`
`sum()`	Calculate sum	`sum([1, 2, 3])`
`max()`	Get maximum	`max([1, 5, 3])`
`min()`	Get minimum	`min([1, 5, 3])`
`sorted()`	Return sorted list	`sorted([3, 1, 2])`

13 Common Beginner Mistakes

Lima kesilapan yang paling kerap berlaku, dengan cara betulkan.

01 GUNA = BUKANNYA ==

INCORRECT
```
if age = 18:
    print("Adult")
```

CORRECT
```
if age == 18:
    print("Adult")
```

02 LUPA INDENTATION

INCORRECT
```
if True:
print("Hello")
```

CORRECT
```
if True:
    print("Hello")
```

03 INPUT IALAH STRING

INCORRECT
```
age = input("Age: ")
print(age + 5)
```

CORRECT
```
age = int(input("Age: "))
print(age + 5)
```

04 LIST INDEX OUT OF RANGE

INCORRECT
```
numbers = [10, 20, 30]

print(numbers[3])  # IndexError
```

CORRECT
```
print(numbers[2])  # 30
```

05 KELIRU PRINT DAN RETURN

`print()` paparkan maklumat pada skrin.

`return` beri value kembali daripada function supaya boleh disimpan atau dikira.

CONTOH
```
def add(a, b):
    return a + b   # value dihantar balik

print(add(5, 3))   # papar 8
```

14 PUT IT ALL TOGETHER

Satu program pendek yang gabungkan beberapa konsep beginner.

MINI PROGRAM
```
def calculate_average(scores):
    return sum(scores) / len(scores)


name = input("Enter student name: ")

scores = [80, 75, 90]

average = calculate_average(scores)

print(f"Student: {name}")
print(f"Average: {average:.2f}")

if average >= 80:
    print("Result: Excellent")
else:
    print("Result: Keep improving")
```

OUTPUT (INPUT: ALI)
```
Enter student name: Ali
Student: Ali
Average: 81.67
Result: Excellent
```

KONSEP YANG DIGUNAKAN

1. Function: `def calculate_average`
2. Parameter: `scores`
3. List: `[80, 75, 90]`
4. Input: `input()`
5. Built-in functions: `sum()`, `len()`
6. Variable: `average`
7. f-string: `f"Average: ..."`
8. Conditional: `if / else`
9. Return value: `return`

Download dan simpan sebelah editor code korang.

Latih contoh dalam editor sendiri. Ubah value, run semula, perhati hasilnya.

yusufaimxn
Technology, Made Understandable.

